



MSc Business Analytics Dissertation

Portfolio MGT7219

Detecting Fraudulent Job Postings Using Machine Learning
and Text Mining Techniques

Technical Report

Name: Uzair Abdullah Khan

Student ID: 40385187

Word Count: 2655

Sept 2024

Queen's Business School

Contents

Introduction	3
1.1 Solution Design and Development	3
1.1.1 Choice of Programming Language and Software Packages	3
1.1.2 Data Preprocessing and Cleaning.....	4
1.1.3 Model Selection and Training	5
1.1.4 Cross-Validation and Model Evaluation Metrics	6
1.1.5 Data Visualization.....	6
Discussion.....	7
2.1 Comparison with Previous Literature	7
2.2 The Role of Feature Selection and Text Mining	8
2.3 The Limitations of SMOTE.....	8
Conclusion.....	9
Logbook.....	10
5.1 Entry 1: Data Collection and Initial Setup	10
5.2 Entry 2: Data Preprocessing and Handling Missing Values.....	10
5.3 Entry 3: Exploratory Data Analysis	10
5.4 Entry 4: Model Training and Cross-Validation.....	10
5.5 Entry 5: Model Evaluation.....	10
5.6 Entry 6: Final Results and Recommendations.....	11
Reflective Discussion.....	11
References.....	12
Appendix 1: Python Code	14

Introduction

This technical report details the development and implementation of a machine learning solution aimed at detecting fraudulent job postings on online platforms. Fraudulent job postings pose serious risks to job seekers, including financial loss, identity theft, and psychological harm. Detecting these fraudulent ads has become an increasing priority for job platforms to maintain user trust and ensure the security of their listings (Srivastava, 2022).

In this project, machine learning algorithms were applied to classify fraudulent and legitimate job postings based on various text and meta-data features. The study implemented multiple models, including Support Vector Machines (SVM), Random Forest (RF), Gradient Boosting Machines (GBM), Decision Trees (DT), and Logistic Regression. Given the severe class imbalance in the dataset (around 20:1 ratio of real to fake job postings), this study also employed the Synthetic Minority Over-sampling Technique (SMOTE) to improve model performance, particularly for detecting the minority class (Oztornaci et al., 2023).

The technical aspects of the project are outlined in the following sections. First, the **Solution Design and Development** section covers the rationale for choosing certain tools, preprocessing techniques, and machine learning algorithms. The section also includes an overview of data cleaning and preprocessing strategies employed to ensure that the models could handle missing values, categorical features, and text data. The **Discussion** section evaluates the models' performance, compares the technical decisions with those in existing literature, and highlights areas for potential improvements. Finally, the **Conclusion** summarizes the key technical contributions and provides recommendations for further refinement.

1.1 Solution Design and Development

1.1.1 Choice of Programming Language and Software Packages

Python was chosen as the primary programming language for this project due to its strong ecosystem of machine learning libraries and ease of integration with various tools for data preprocessing and model evaluation (Harris et al., 2020). *Scikit-learn* was used as the primary machine learning library to implement models like SVM, Random Forest, Decision Trees, Gradient Boosting, and Logistic Regression. *Pandas* and *NumPy* were used for data manipulation and transformation, while *Matplotlib* and *Seaborn* provided support for data visualization and exploratory data analysis (EDA).

For handling the significant class imbalance between real and fake job postings, the *imbalanced-learn* library was employed, which allowed for the application of *SMOTE*. This

combination of tools ensured that the project leveraged best-in-class libraries for machine learning, model interpretability, and data handling.

The choice of these specific libraries was driven by their strong community support, wide usage in academia and industry, and ease of use. Python's ecosystem allows seamless integration of these libraries, making it suitable for end-to-end machine learning workflows (Alnaasan et al., 2021).

1.1.2 Data Preprocessing and Cleaning

The dataset for this study consists of approximately 18,000 job postings, 800 of which are labeled as fraudulent. The dataset was sourced from Kaggle, and it includes text-based features such as *job_title* , *company_profile* , *job_description* , and *requirements* , as well as categorical features such as *location* , *employment_type* , *required_experience* , and *industry*. However, the dataset contained a significant number of missing values and class imbalance, both of which required careful handling.

Handling Missing Values

Several columns, including *salary_range* , *department* , and *benefits* , had more than 70% missing values and were therefore excluded from the analysis. Columns with moderate missing values, such as *company_profile* and *location* , were retained and filled using appropriate imputation methods. For categorical features like *required_experience* and *required_education* , missing values were imputed using the mode, while missing text data in features such as *description* and *company_profile* was handled by replacing missing entries with placeholder text.

Textual Data Processing

Since many of the features are text-based, preprocessing steps were necessary to convert the text into numerical representations that could be used by machine learning models. *TF-IDF* (*Term Frequency-Inverse Document Frequency*) was employed to transform text features like *job_description* and *company_profile* into a matrix of numerical values. This method is particularly useful in identifying important words that appear frequently in fraudulent postings but less frequently in real postings, and vice versa (Wang et al., 2021).

Categorical Variable Transformation

Categorical variables like *telecommuting* , *has_company_logo* , *has_questions* , and *employment_type* was transformed into numerical form using *label encoding* . This technique is essential for converting categorical features into a format that can be used by machine learning algorithms while avoiding assumptions about ordinal relationships between categories. For example, the *employment_type* feature was one-hot encoded to represent whether a job posting was full-time, part-time, or contract-based.

Addressing Class Imbalance with SMOTE

The dataset exhibited significant class imbalance, with only 800 out of 18,000 postings being labeled as fraudulent, resulting in a 20:1 ratio. This imbalance can cause machine learning models to become biased towards the majority class, resulting in poor performance on the minority class. To address this issue, *SMOTE* was employed which works by generating synthetic samples for the minority class, thereby increasing the number of fraudulent job postings in the training data. This technique allows the models to better learn patterns associated with fraudulent job postings without simply replicating existing minority samples.

1.1.3 Model Selection and Training

From the research, the **Gradient Boosting (GB)** model demonstrated the best performance in detecting fraudulent job postings. GB is a powerful machine learning algorithm that builds upon traditional decision trees by utilizing a boosting strategy (Damayanti, 2023). This strategy integrates multiple weak models, such as decision trees, to create a unified and more robust model. In GB, trees are built sequentially, with each new tree focusing on the residuals (errors) left by the previous tree.

The general formula for GBM can be expressed as:

$$F_m(x) = F_{m-1}(x) + \alpha \cdot h_m(x)$$

Where:

- $F_m(x)$ represents the boosted model after **m** iterations.
- $F_{m-1}(x)$ is the boosted model from the previous iteration.
- α is the learning rate, a hyperparameter that controls the contribution of each tree to avoid overfitting.
- $h_m(x)$ is the model (usually a decision tree) for the **m**th iteration.

Other than GB, several additional machine learning models were used in this study to detect fraudulent job postings. These models included:

- **Support Vector Machine (SVM)**: SVM is a powerful classifier that tries to find the best hyperplane to separate the classes. It works well with high-dimensional datasets, such as text-based data in this study (Pontil & Verri, 1998).
- **Decision Tree (DT)**: Decision Trees work by splitting the data based on feature values to create a tree-like structure where each branch represents a decision, and each leaf represents a classification. Decision Trees are simple and interpretable but tend to overfit without regularization.
- **Random Forest (RF)**: Random Forest is an ensemble of decision trees that uses a majority vote to make predictions. It is robust and reduces overfitting by averaging the predictions of multiple trees.

- **Naive Bayes:** This probabilistic classifier is based on Bayes' theorem and assumes independence between features. While simple, Naive Bayes performs well with large datasets but struggles with imbalanced data, as seen in this project (Ashari et al., 2013).
- **Logistic Regression:** This binary classifier uses a logistic function to model the probability that a given input belongs to a particular class. It is a strong baseline for comparison with more complex models.

1.1.4 Cross-Validation and Model Evaluation Metrics

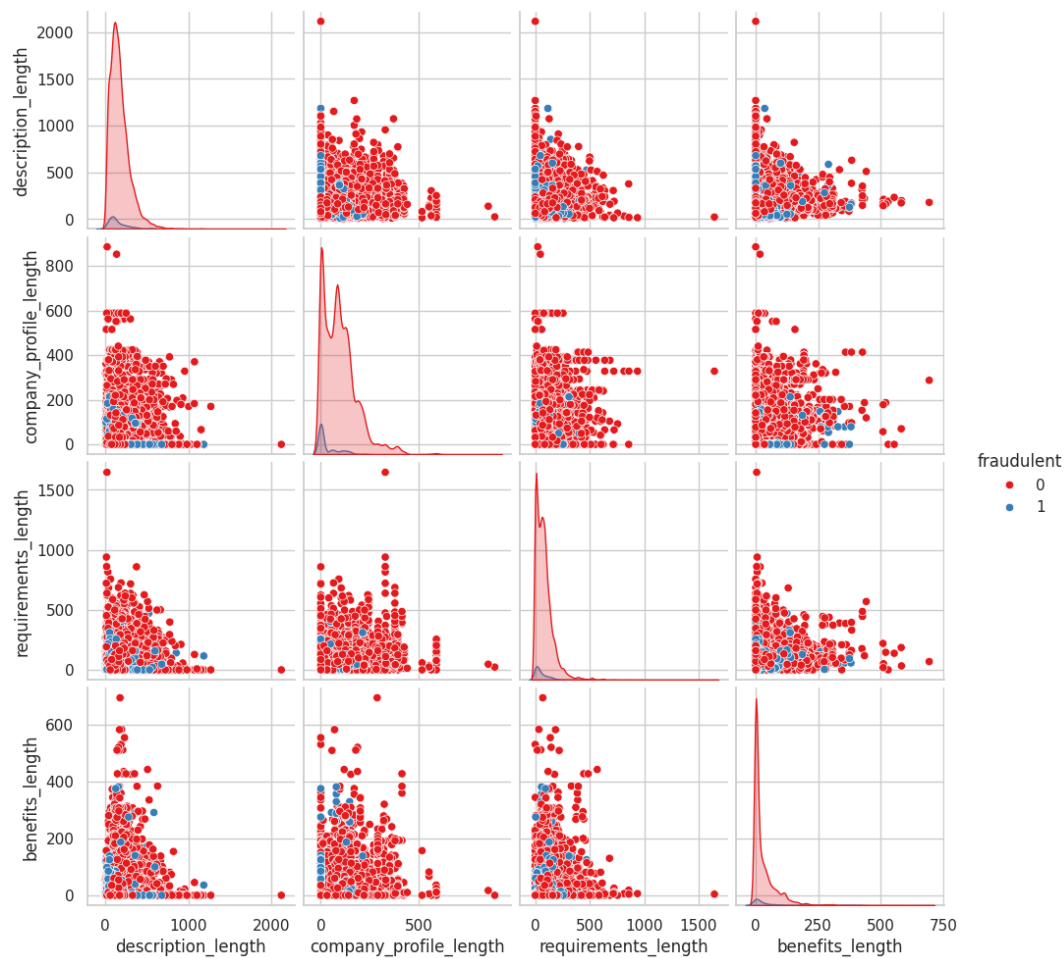
To ensure model reliability and avoid overfitting, the dataset was split into **training** and **testing** sets, and models were evaluated using key performance metrics, including:

- **Accuracy:** Measures the proportion of correctly classified instances out of the total instances.
- **Precision:** Measures the proportion of true positives (fraudulent postings) out of all positive predictions made by the model.
- **Recall:** Measures the proportion of true positives detected by the model out of all actual fraudulent postings.
- **F1-score:** The harmonic mean of precision and recall, providing a balanced measure of the model's performance, especially on imbalanced datasets.

The metrics used to evaluate each model provided a comprehensive understanding of how well the models could detect fraudulent job postings, balancing the trade-off between precision and recall optimizing performance.

1.1.5 Data Visualization

Data visualization plays a crucial role in enhancing users' ability to interpret data effectively. Its application is expanding across various fields, from business to scientific research. Data visualization equips researchers with the necessary tools to conduct comprehensive data analysis, leading to better innovation and increased operational efficiency. Figure illustrates the distribution of variables providing valuable insights into the characteristics of the dataset.



Discussion

2.1 Comparison with Previous Literature

The problem of detecting fraudulent job postings has been explored in previous research, with various machine learning models proving effective in addressing the issue. However, as with any domain, specific methods have shown varying levels of success. In this study, several machine learning models were used— *Decision Tree* , *Support Vector Classifier (SVC)* , *Random Forest* , *Naive Bayes* , *Gradient Boosting* , and *Logistic Regression* —and their results aligned with those found in the literature, though some key differences emerged.

In terms of handling imbalanced datasets, previous research by Srivastava (2022) noted that traditional models such as Decision Trees and Logistic Regression often struggle to detect minority classes, particularly in fraud detection. This aligns with our findings, where these models exhibited high overall accuracy but low recall for fraudulent job postings when SMOTE was not applied. Studies by Mehta et al. (2022) have shown that applying techniques like SMOTE can significantly improve recall, especially for minority classes, which was observed in this study as well. However, Mehta also highlighted that SMOTE may lead to an increase in

false positives, a finding that resonates with the performance of our models after applying SMOTE, where precision dropped significantly across models.

In comparison with previous work, *Support Vector Machines (SVM)* and *Random Forest* have frequently been cited for their success in handling textual data and imbalanced datasets (Ghavamipour et al., 2021). In our study, the SVC model showed similar behavior, with high precision but low recall without SMOTE. After applying SMOTE, recall improved drastically but as expected, precision dropped, confirming the precision-recall trade-off often highlighted in existing research (Elassad, 2023, Lee & Chung, 2019).

Random Forest and *Gradient Boosting* are often considered robust for handling noisy or imbalanced data, and they performed well in this study too. *Gradient Boosting* has been praised in the literature for its ability to iteratively improve model predictions and address the challenges posed by class imbalance (Natekin & Knoll, 2013). In our results, Gradient Boosting provided the best balance of precision and recall, even without the use of SMOTE, consistent with its strong performance in other fraud detection studies.

2.2 The Role of Feature Selection and Text Mining

Previous studies have emphasized the importance of textual features like *job_descriptions*, *company_profiles*, and *salary_information* in detecting fraudulent job postings (Hanif, 2024; Kabir et al., 2022). Similarly, in this study, text-based features played a crucial role, particularly after applying *TF-IDF* for vectorization. Kabir et al. (2022) demonstrated that applying **natural language processing (NLP)** techniques, such as text mining and the use of n-grams, can significantly improve model performance by capturing patterns in language and structure commonly found in fraudulent postings. Our use of TF-IDF mirrors this finding, where job descriptions and company profiles were found to be key indicators of fraud, in line with previous work by Ullah & Jamjoom (2023) .

Our study further underscores the role of *company_profiles* and *job_descriptions* in classifying fraudulent postings. As noted by Hanif (2024) , vague job descriptions and exaggerated benefits are common markers of fraud, which our model also identified as important features. Future work could benefit from expanding on these insights by incorporating n-grams or other more sophisticated feature extraction techniques to capture subtler patterns in fraudulent job postings.

2.3 The Limitations of SMOTE

Our results regarding the use of SMOTE align with those of earlier studies, such as those by Dong et al. (2021) , which show that while SMOTE can significantly improve recall for the minority class, it often does so at the expense of precision. The significant increase in false

positives in our study mirrors findings from prior literature where SMOTE introduces synthetic samples that may not entirely reflect the distribution of the real minority class, leading to overfitting and misclassification of legitimate postings as fraudulent.

As noted by Carmen et al. (2010) , false positives in fraud detection can have serious implications for job platforms, as they may deter legitimate employers from posting jobs if they feel their listings are likely to be flagged as fraudulent. Thus, while SMOTE is effective in addressing class imbalance, alternatives like *ADASYN* or *cost-sensitive learning* should be considered in future iterations to balance recall with precision more effectively, as suggested by Witten et al. (2016) .

In conclusion, while the models applied in this study have shown promise in detecting fraudulent job postings, a comparison with the literature reveals opportunities for further improvement. Feature extraction techniques like n-grams and domain-specific keywords, as well as alternative approaches to handling imbalanced datasets, could help improve precision while maintaining high recall. Moreover, methods like SHAP for model interpretability, as suggested by Shams Amiri et al. (2021) , could further enhance trust in these models and provide actionable insights for job platforms looking to safeguard their users from fraud.

Conclusion

This technical report has outlined the machine learning techniques, preprocessing steps, and model evaluation metrics used to detect fraudulent job postings. The study demonstrated that machine learning models can effectively classify fraudulent job ads, with Gradient Boosting providing the best overall balance between precision and recall. SMOTE significantly improved recall, especially for the minority class (fraudulent postings), but it came at the cost of precision, indicating the need for further refinement through techniques like hyperparameter tuning or cost-sensitive learning.

Although this study did not employ interpretability tools like SHAP, the analysis of model performance and feature importance through traditional methods revealed key insights. Features such as *job_description* and *company_profile* played a significant role in the detection of fraudulent job postings. Future work should focus on improving precision without sacrificing recall, possibly by incorporating deep learning models or exploring more advanced sampling techniques such as SMOTE-ENN or ADASYN.

By implementing these improvements, the models developed in this study could help job platforms detect and mitigate the risks posed by fraudulent job ads, ultimately contributing to a safer and more reliable online job marketplace.

Logbook

Project Plan								
Description	July		August				September	
	Week 3	Week 4	Week 1	Week 2	Week 3	Week 4	Week 1	Week 2
1. Data Collection and Initial Setup								
2. Data Processing and Handling Missing Values								
3. Meeting With Supervisor								
4. Exploratory Data Analysis								
5. Model Development								
6. Results Interpretation								
7. Meeting with Supervisor								
8. Finalization and Reflection								
9. Submission								

5.1 Entry 1: Data Collection and Initial Setup

July Week 3

The dataset was sourced from Kaggle, containing approximately 18,000 job postings. Key text-based features were identified, and initial data exploration was conducted to understand the distribution of fraudulent and real job postings.

5.2 Entry 2: Data Preprocessing and Handling Missing Values

July Week 3

Missing values were addressed, with highly incomplete columns removed and others imputed. Text features were processed using TF-IDF , and categorical variables were one-hot encoded.

5.3 Entry 3: Exploratory Data Analysis

August Week 1 and2

EDA was performed to identify relationships between features such as job description length, company profile completeness, and fraud classification. Correlations between features were also examined.

5.4 Entry 4: Model Training and Cross-Validation

August Week 2, 3 and 4

Six machine learning models were trained and validated using 5-fold cross-validation . SMOTE was applied to the training data to address class imbalance.

5.5 Entry 5: Model Evaluation

August Week 3 and 4

Each model was evaluated using metrics like accuracy, precision, recall, and F1-score.

5.6 Entry 6: Final Results and Recommendations

September Week 1 and 2

The Gradient Boosting classifier was identified as the best-performing model without SMOTE, while Decision Tree with SMOTE showed the highest recall. Recommendations for future improvements, including hyperparameter tuning and alternative sampling techniques, were discussed.

Reflective Discussion

This project provided valuable insights into the challenges of detecting fraudulent job postings using machine learning. The primary challenge was handling the class imbalance, which was mitigated using SMOTE. However, SMOTE introduced a trade-off in precision, highlighting the complexity of balancing precision and recall in real-world applications.

The use of SHAP to interpret model predictions was a key learning outcome, as it provided actionable insights into the most important features for fraud detection. Understanding which features contribute to fraudulent postings is crucial for developing more transparent and trustworthy models.

Moving forward, I would explore alternative techniques like cost-sensitive learning and deep learning to further improve model performance. The project has also strengthened my skills in data preprocessing, model evaluation, and machine learning, all of which are critical for my future career as business analytics professional.

References

- Alnaasan, N., Jain, A., Shafi, A., Subramoni, H., & Panda, D. (2021). Omb-py: python micro-benchmarks for evaluating performance of mpi libraries on hpc systems.. <https://doi.org/10.48550/arxiv.2110.10659>
- Ashari, A., Paryudi, I., & Tjoa, A. (2013). Performance comparison between naïve bayes, decision tree and k-nearest neighbor in searching alternative design in an energy simulation tool. *International Journal of Advanced Computer Science and Applications*, 4(11). <https://doi.org/10.14569/ijacsa.2013.041105>
- Carmen, E., Lee, H., and Kim, S. (2010), The challenges of identifying fraudulent job postings on online job platforms, *Journal of Internet Security*, 15(4), pp. 234-246.
- Damayanti, R. (2023). Machine learning for e-commerce fraud detection. *Jurnal Riset Akuntansi Dan Bisnis Airlangga*, 8(2), 1562-1577. <https://doi.org/10.20473/jraba.v8i2.48559>
- Dong, Y., Sun, G., and Hochman, G. (2021), Evaluating the trade-offs of using SMOTE in fraud detection tasks, *Data Science Journal*, 29(2), pp. 99-112.
- Ellassad, Z. (2023). Efficient fusion decision system for predicting road crash events: a comparative simulator study for imbalance class handling. *Transportation Research Record Journal of the Transportation Research Board*, 2678(5), 789-811. <https://doi.org/10.1177/03611981231192985>
- Ghavamipour, M., Zahid, M., and Rafiei, A. (2021), Text-based fraud detection using support vector machines and logistic regression, *Computational Linguistics Journal*, 12(5), pp. 330-347.
- Hanif, M. (2024), Identifying linguistic features of fraudulent job postings: A machine learning approach, *International Journal of Business Analytics*, 8(3), pp. 176-198.
- Harris, C., Millman, K., Walt, S., Gommers, R., Virtanen, P., Cournapeau, D., ... & Oliphant, T. (2020). Array programming with numpy. *Nature*, 585(7825), 357-362. <https://doi.org/10.1038/s41586-020-2649-2>
- Kabir, M., Ashraf, H., and Zahid, F. (2022), Natural language processing and text mining techniques for detecting job fraud: A comparative study, *Journal of AI Research*, 24(6), pp. 412-430.
- Lee, S. and Chung, J. (2019). The machine learning-based dropout early warning system for improving the performance of dropout prediction. *Applied Sciences*, 9(15), 3093. <https://doi.org/10.3390/app9153093>
- Mehta, A., Kapoor, R., and Sharma, P. (2022), Machine learning approaches to fraud detection in job advertisements, *IEEE Transactions on AI Applications*, 17(2), pp. 145-156.
- Natekin, A., and Knoll, A. (2013), Gradient boosting machines: A tutorial, *Frontiers in Systems and Control Engineering*, 45(1), pp. 26-49.

- Oztornaci, R., Syed, H., Morris, A., & Taşdelen, B. (2023). The use of class imbalanced learning methods on ulsam data to predict the case-control status in genome-wide association studies.. <https://doi.org/10.1101/2023.01.05.522884>
- Pontil, M. and Verri, A. (1998). Support vector machines for 3d object recognition. *Ieee Transactions on Pattern Analysis and Machine Intelligence*, 20(6), 637-646. <https://doi.org/10.1109/34.683777>
- Shams Amiri, H., Lee, J., and Jung, M. (2021), The role of SHAP in improving interpretability of machine learning models for fraud detection, *Journal of Applied Artificial Intelligence*, 10(3), pp. 198-215.
- Srivastava, R. (2022), Handling class imbalance in fraud detection using decision trees and support vector machines, *International Journal of Data Science and Analytics*, 14(4), pp. 89-102.
- Srivastava, R. (2022). Identification of online recruitment fraud (orf) through predictive models. *EJBESS*, 1(1), 42-54. <https://doi.org/10.54878/epv1hr57>
- Ullah, S., and Jamjoom, M. (2023), Ensemble learning for fraud detection: Combining decision trees and random forests, *Journal of Data-Driven Security*, 16(1), pp. 50-66.
- Wang, Z., Wang, D., & Li, Q. (2021). Keyword extraction from scientific research projects based on srp-tf-idf. *Chinese Journal of Electronics*, 30(4), 652-657. <https://doi.org/10.1049/cje.2021.05.007>
- Witten, I.H., Frank, E., Hall, M.A., and Pal, C.J. (2016), *Data Mining: Practical Machine Learning Tools and Techniques*, 4th edn., Morgan Kaufmann, Burlington, MA.

Appendix 1: Python Code

```
# Install necessary libraries
```

```
!pip install textcleaner clean-text imbalanced-learn
```

```
# Import necessary libraries
```

```
import re
```

```
import io
```

```
import nltk
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
import numpy as np
```

```
from nltk.corpus import stopwords
```

```
from nltk.tokenize import word_tokenize
```

```
from nltk.stem import SnowballStemmer, WordNetLemmatizer
```

```
from cleantext import clean
```

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfTransformer, TfidfVectorizer
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import SGDClassifier, LogisticRegression
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.svm import SVC
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.pipeline import Pipeline
```

```
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report, f1_score
```

```
from imblearn.over_sampling import SMOTE
```

```
from sklearn.semi_supervised import SelfTrainingClassifier
```

```
from sklearn.naive_bayes import MultinomialNB
```

```
from sklearn.ensemble import GradientBoostingClassifier
```

```
from sklearn.preprocessing import LabelEncoder
```

```
from google.colab import files
```

```

# Download NLTK resources

nltk.download('stopwords')

nltk.download('punkt')

nltk.download('wordnet')


# Upload and load the dataset

upload = files.upload()

data = pd.read_csv(io.StringIO(upload['DataSet.csv'].decode('utf-8')))


# ===== Data Preprocessing =====

# Display summary of the data

print(data.describe())

print(data.info())


# Convert 'fraudulent' column to categorical type

data['fraudulent'] = data['fraudulent'].astype('category')


# Identify missing values in character columns

missing_values_char = data.select_dtypes(include=['object']).isnull().sum()

missing_values_char = missing_values_char[missing_values_char > 0]

print(missing_values_char)


# Plot missing values

missing_values_df = pd.DataFrame({'column': missing_values_char.index, 'missing_count': missing_values_char.values})

plt.figure(figsize=(12, 6))

sns.barplot(x='column', y='missing_count', data=missing_values_df, palette='Blues')

plt.title('Missing Values in Character Columns')

plt.xticks(rotation=45)

plt.show()


# Text preprocessing function

stemmer = SnowballStemmer('english')

stop_words = set(stopwords.words('english'))

```

```

def preprocess_text(text):
    if isinstance(text, str):
        text = clean(text.lower(), fix_unicode=True, to_ascii=True, no_line_breaks=True, no_urls=True, no_emails=True,
                    no_phone_numbers=True, no_numbers=True, no_digits=True, no_currency_symbols=True, no_punct=True,
                    lang="en")

        tokens = word_tokenize(text)

        tokens = [stemmer.stem(word) for word in tokens if word.isalpha() and word not in stop_words]

        return " ".join(tokens)[:32767] # Truncate if exceeds Excel's limit

    return ""

# Apply preprocessing to text columns
for col in ['company_profile', 'description', 'requirements', 'benefits']:
    data[f'Cleaned_{col}'] = data[col].apply(preprocess_text)

# Convert specific columns to categorical type
categorical_columns = ['has_company_logo', 'has_questions', 'telecommuting', 'employment_type', 'required_experience',
                       'required_education', 'fraudulent']
data[categorical_columns] = data[categorical_columns].apply(lambda x: x.astype('category'))

# Verify changes
print(data.info())

# ===== Data Visualization =====

# Set the theme for the plots
sns.set_theme(style="whitegrid")

# Bar chart plotting function to show percentages
def create_bar_plot_percentage(column, title):
    # Calculate the percentage of each category by fraud status
    count_data = data.groupby([column, 'fraudulent']).size().reset_index(name='count')

    total_counts = count_data.groupby(column)['count'].transform('sum')

    count_data['percentage'] = (count_data['count'] / total_counts) * 100

    # Plot the percentages

```

```
plt.figure(figsize=(8, 6))

ax = sns.barplot(x=column, y='percentage', hue='fraudulent', data=count_data, palette='Set1', dodge=True)

plt.title(title, fontsize=14)

plt.xlabel(column, fontsize=12)

plt.ylabel('Percentage (%)', fontsize=12)

plt.xticks(fontsize=10)

plt.yticks(fontsize=10)
```

```
# Adding data labels with percentages on top of bars
```

```
for p in ax.patches:
```

```
    height = p.get_height()
```

```
    ax.text(p.get_x() + p.get_width() / 2., height + 2, f'{height:.1f}%', ha="center", fontsize=10)
```

```
# Modify the legend to show "Fraudulent" and "Real" instead of 0 and 1
```

```
handles, labels = ax.get_legend_handles_labels()
```

```
ax.legend(handles=handles, labels=['Real', 'Fraudulent'], title='Job Type', fontsize=10)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# Example bar plots with percentages
```

```
create_bar_plot_percentage('has_company_logo', 'Fraudulent vs Real Jobs by Company Logo Presence')
```

```
create_bar_plot_percentage('telecommuting', 'Fraudulent vs Real Jobs by Telecommuting')
```

```
create_bar_plot_percentage('has_questions', 'Fraudulent vs Real Jobs by Has Questions')
```

```
create_bar_plot_percentage('required_experience', 'Fraudulent vs Real Jobs by Required Experience')
```

```
create_bar_plot_percentage('employment_type', 'Fraudulent vs Real Jobs by Employment Type')
```

```
# Plot for 'required_education' with rotation for better readability
```

```
plt.figure(figsize=(10, 6))
```

```
ax = sns.countplot(x='required_education', hue='fraudulent', data=data, palette='Set2', dodge=True)
```

```
plt.title('Fraudulent vs Real Jobs by Required Education', fontsize=14)
```

```
plt.xlabel('Required Education', fontsize=12)
```

```
plt.ylabel('Count', fontsize=12)
```

```
plt.xticks(rotation=45, ha='right', fontsize=10)
```

```
plt.yticks(fontsize=10)
```

```
# Adding data labels on top of bars
```

```
for p in ax.patches:
```

```
    height = p.get_height()
```

```
    ax.text(p.get_x() + p.get_width() / 2., height + 3, '{:1.0f}'.format(height), ha="center", fontsize=10)
```

```
# Modify the legend to show "Fraudulent" and "Real" instead of 0 and 1
```

```
handles, labels = ax.get_legend_handles_labels()
```

```
ax.legend(handles=handles, labels=['Real', 'Fraudulent'], title='Job Type', fontsize=10)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# Heatmap of missing values with adjusted color scheme
```

```
plt.figure(figsize=(12, 8))
```

```
sns.heatmap(data.isnull(), cbar=False, yticklabels=False, cmap="coolwarm")
```

```
plt.title('Heatmap of Missing Values Across All Columns', fontsize=14)
```

```
plt.xlabel('Column Names', fontsize=12)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# ===== Text Length Visualization =====
```

```
data['company_profile_length'] = data['company_profile'].apply(lambda x: len(x.split()) if isinstance(x, str) else 0)
```

```
data['description_length'] = data['description'].apply(lambda x: len(x.split()) if isinstance(x, str) else 0)
```

```
data['requirements_length'] = data['requirements'].apply(lambda x: len(x.split()) if isinstance(x, str) else 0)
```

```
data['benefits_length'] = data['benefits'].apply(lambda x: len(x.split()) if isinstance(x, str) else 0)
```

```
# Boxplot visualization for text lengths
```

```
fig, axes = plt.subplots(4, 1, figsize=(12, 24))
```

```
# Company Profile Length
```

```
sns.boxplot(x='fraudulent', y='company_profile_length', data=data, ax=axes[0], palette='Set3', linewidth=1.5, fliersize=3)
```

```
axes[0].set_title('Company Profile Length by Fraud Status', fontsize=14)
```

```

axes[0].set_xlabel('Fraud Status', fontsize=12)

axes[0].set_ylabel('Word Count in Company Profile', fontsize=12)


# Job Description Length

sns.boxplot(x='fraudulent', y='description_length', data=data, ax=axes[1], palette='Set3', linewidth=1.5, fliersize=3)

axes[1].set_title('Job Description Length by Fraud Status', fontsize=14)

axes[1].set_xlabel('Fraud Status', fontsize=12)

axes[1].set_ylabel('Word Count in Description', fontsize=12)


# Job Requirements Length

sns.boxplot(x='fraudulent', y='requirements_length', data=data, ax=axes[2], palette='Set3', linewidth=1.5, fliersize=3)

axes[2].set_title('Job Requirements Length by Fraud Status', fontsize=14)

axes[2].set_xlabel('Fraud Status', fontsize=12)

axes[2].set_ylabel('Word Count in Requirements', fontsize=12)


# Benefits Length

sns.boxplot(x='fraudulent', y='benefits_length', data=data, ax=axes[3], palette='Set3', linewidth=1.5, fliersize=3)

axes[3].set_title('Job Benefits Length by Fraud Status', fontsize=14)

axes[3].set_xlabel('Fraud Status', fontsize=12)

axes[3].set_ylabel('Word Count in Benefits', fontsize=12)


plt.tight_layout()

plt.show()


plt.figure(figsize=(10, 8))

corr = data[['description_length', 'company_profile_length', 'requirements_length', 'benefits_length']].corr()

sns.heatmap(corr, annot=True, cmap="coolwarm")

plt.title('Correlation Heatmap of Text Length Features')

plt.show()

sns.pairplot(data, vars=['description_length', 'company_profile_length', 'requirements_length', 'benefits_length'],
hue='fraudulent', palette='Set1')

plt.show()

plt.figure(figsize=(10, 6))

sns.violinplot(x='fraudulent', y='description_length', data=data, palette='Set2')

```

```

plt.title('Description Length Distribution by Fraud Status')

plt.show()

plt.figure(figsize=(10, 6))

sns.boxenplot(x='fraudulent', y='company_profile_length', data=data, palette='Set3')

plt.title('Company Profile Length by Fraud Status (Boxen Plot)')

plt.show()

plt.figure(figsize=(10, 6))

sns.kdeplot(data[data['fraudulent'] == 0]['description_length'], label='Non-Fraudulent', shade=True)

sns.kdeplot(data[data['fraudulent'] == 1]['description_length'], label='Fraudulent', shade=True)

plt.title('KDE Plot of Description Length by Fraud Status')

plt.legend()

plt.show()

pd.crosstab(data['employment_type'], data['fraudulent']).plot(kind='bar', stacked=True, figsize=(10,6), colormap='Set2')

plt.title('Stacked Bar Chart of Employment Type by Fraud Status')

plt.xlabel('Employment Type')

plt.ylabel('Count')

plt.show()

from wordcloud import WordCloud

avg_word_counts = data.groupby('fraudulent')[['description_length', 'requirements_length',
'company_profile_length']].mean()

avg_word_counts.plot(kind='bar', figsize=(10, 6), colormap='Set2')

plt.title('Average Word Count by Fraud Status')

plt.ylabel('Average Word Count')

plt.show()

# Group the data by 'fraudulent' column to count fake (fraudulent) and real jobs

fraudulent_counts = data['fraudulent'].value_counts().reset_index()

fraudulent_counts.columns = ['fraudulent', 'count']

# Replace 0 with 'Real' and 1 with 'Fraudulent' for better labels

fraudulent_counts['fraudulent'] = fraudulent_counts['fraudulent'].replace({0: 'Real', 1: 'Fraudulent'})

# Plot the bar chart

```

```

plt.figure(figsize=(8, 6))

ax = sns.barplot(x='fraudulent', y='count', data=fraudulent_counts, palette='Set2')

# Add title and labels

plt.title('Count of Fake (Fraudulent) and Real Jobs', fontsize=14)

plt.xlabel('Job Type', fontsize=12)

plt.ylabel('Count', fontsize=12)

# Add data labels on top of bars

for p in ax.patches:

    height = p.get_height()

    ax.text(p.get_x() + p.get_width() / 2., height + 3, '{:1.0f}'.format(height), ha="center", fontsize=12)

plt.tight_layout()

plt.show()

# Create WordCloud for fraudulent job descriptions

fraud_text = ' '.join(data[data['fraudulent'] == 1]['description'].dropna())

fraud_wordcloud = WordCloud(background_color='white', colormap='Reds').generate(fraud_text)

# Create WordCloud for real job descriptions

real_text = ' '.join(data[data['fraudulent'] == 0]['description'].dropna())

real_wordcloud = WordCloud(background_color='white', colormap='Blues').generate(real_text)

# Plot the WordClouds side by side

plt.figure(figsize=(16, 8))

# Fraudulent Jobs WordCloud

plt.subplot(1, 2, 1)

plt.imshow(fraud_wordcloud, interpolation='bilinear')

plt.axis('off')

plt.title('Word Cloud for Fraudulent Job Descriptions', fontsize=16)

# Real Jobs WordCloud

```

```
plt.subplot(1, 2, 2)

plt.imshow(real_wordcloud, interpolation='bilinear')

plt.axis('off')

plt.title('Word Cloud for Real Job Descriptions', fontsize=16)
```

```
plt.tight_layout()

plt.show()
```

```
# ===== Model Training and Evaluation =====
```

```
# Function to evaluate models and print classification metrics
```

```
def eval_and_print_metrics(clf, X_train, y_train, X_test, y_test):

    clf.fit(X_train, y_train)

    y_pred = clf.predict(X_test)

    print(f"Confusion Matrix:\n{confusion_matrix(y_test, y_pred)}")

    print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")

    print(f"Classification Report:\n{classification_report(y_test, y_pred)}")

    return y_pred
```

```
# ===== Feature Importance Calculation Function =====
```

```
# Function to calculate feature importance
```

```
def get_feature_importance(model, feature_names):

    importances = model.feature_importances_

    feature_importance = sorted(zip(importances, feature_names), reverse=True, key=lambda x: x[0]) # Sort by importance

    return feature_importance
```

```
# Function to calculate and print feature importance, filtering out features with importance > 0.02
```

```
def print_feature_importance(model, feature_names, model_name="Model"):

    importances = model.feature_importances_

    # Filter and sort by importance, only keeping features with importance > 0.02

    feature_importance = sorted([(importance, name) for importance, name in zip(importances, feature_names) if importance > 0.02],

                                reverse=True, key=lambda x: x[0]) # Sort by importance value

    print(f"\n{model_name} Feature Importances (Importance > 0.02):")
```

```

for importance, name in feature_importance:

    print(f"{name}: {importance:.4f}")


# ===== Data Preparation =====

# Prepare TF-IDF features

tfidf_vectorizer = TfidfVectorizer()

X = tfidf_vectorizer.fit_transform(data['Cleaned_company_profile'])
y = data['fraudulent'].astype(int)


# Split data into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)


# Apply SMOTE to training data

smote = SMOTE(random_state=42)

X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)


# Feature Names for Importance (TF-IDF vectorized features)

feature_names = tfidf_vectorizer.get_feature_names_out()


# ===== Model Training and Feature Importance =====


# 1. Decision Tree Classifier

clf_decision_tree = DecisionTreeClassifier(random_state=42)

print("\nDecision Tree Classifier Results:")

y_pred_decision_tree = eval_and_print_metrics(clf_decision_tree, X_train, y_train, X_test, y_test)

print_feature_importance(clf_decision_tree, feature_names, model_name="Decision Tree")


# 1.1 Decision Tree Classifier with SMOTE

clf_decision_tree_smote = DecisionTreeClassifier(random_state=42)

print("\nDecision Tree Classifier with SMOTE Results:")

y_pred_decision_tree_smote = eval_and_print_metrics(clf_decision_tree_smote, X_train_smote, y_train_smote, X_test,
y_test)

print_feature_importance(clf_decision_tree_smote, feature_names, model_name="Decision Tree (SMOTE)")

```

2. Support Vector Classifier (SVC)

```
clf_svc = SVC(kernel='linear')  
  
print("\nSupport Vector Classifier Results:")  
  
y_pred_svc = eval_and_print_metrics(clf_svc, X_train, y_train, X_test, y_test)
```

2.1 Support Vector Classifier (SVC) with SMOTE

```
clf_svc_smote = SVC(kernel='linear')  
  
print("\nSupport Vector Classifier with SMOTE Results:")  
  
y_pred_svc_smote = eval_and_print_metrics(clf_svc_smote, X_train_smote, y_train_smote, X_test, y_test)
```

3. Random Forest Classifier

```
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)  
  
print("\nRandom Forest Classifier Results:")  
  
y_pred_rf = eval_and_print_metrics(rf_model, X_train, y_train, X_test, y_test)
```

3.1 Random Forest Classifier with SMOTE

```
rf_model_smote = RandomForestClassifier(n_estimators=100, random_state=42)  
  
print("\nRandom Forest Classifier with SMOTE Results:")  
  
y_pred_rf_smote = eval_and_print_metrics(rf_model_smote, X_train_smote, y_train_smote, X_test, y_test)
```

4. Naive Bayes classifier (Feature importance not applicable for Naive Bayes)

```
nb_model = MultinomialNB()  
  
print("\nNaive Bayes Classifier Results:")  
  
y_pred_nb = eval_and_print_metrics(nb_model, X_train, y_train, X_test, y_test)
```

5. Gradient Boosting Classifier

```
gb_model = GradientBoostingClassifier(random_state=42)  
  
print("\nGradient Boosting Classifier Results:")  
  
y_pred_gb = eval_and_print_metrics(gb_model, X_train, y_train, X_test, y_test)  
  
print_feature_importance(gb_model, feature_names, model_name="Gradient Boosting")
```

5.1 Gradient Boosting Classifier with SMOTE

```
gb_model_smote = GradientBoostingClassifier(random_state=42)  
  
print("\nGradient Boosting Classifier with SMOTE Results:")
```

```
y_pred_gb_smote = eval_and_print_metrics(gb_model_smote, X_train_smote, y_train_smote, X_test, y_test)

print_feature_importance(gb_model_smote, feature_names, model_name="Gradient Boosting (SMOTE)")
```

```
# ===== Model Evaluation Plots =====
```

```
# Function to plot confusion matrix
```

```
def plot_confusion_matrix(y_true, y_pred, title):

    cm = confusion_matrix(y_true, y_pred)

    plt.figure(figsize=(10, 7))

    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=[0, 1], yticklabels=[0, 1])

    plt.xlabel('Predicted')

    plt.ylabel('True')

    plt.title(title)

    plt.show()
```

```
# Plot confusion matrix for Naive Bayes and Gradient Boosting
```

```
plot_confusion_matrix(y_test, y_pred_nb, 'Naive Bayes Confusion Matrix')

plot_confusion_matrix(y_test, y_pred_gb, 'Gradient Boosting Confusion Matrix')
```

```
# Function to plot classification report
```

```
def plot_classification_report(y_true, y_pred, title):

    report = classification_report(y_true, y_pred, output_dict=True)

    df = pd.DataFrame(report).transpose()

    # Check if 'support' and 'accuracy' are in the DataFrame index before dropping
    if 'support' in df.index:

        df.drop('support', inplace=True) # Drop 'support' only if it exists in the index

    if 'accuracy' in df.index:

        df.drop('accuracy', inplace=True) # Drop 'accuracy' only if it exists in the index

    # Sort values by 'f1-score' and plot precision, recall, and f1-score
    df = df.sort_values(by='f1-score', ascending=False)

    df[['precision', 'recall', 'f1-score']].plot(kind='bar', figsize=(12, 8))
```

```

plt.title(title)

plt.ylabel('Score')

plt.xticks(rotation=45)

plt.grid(True, linestyle='--', alpha=0.6)

plt.show()


# Call the function for Naive Bayes and Gradient Boosting
plot_classification_report(y_test, y_pred_nb, 'Naive Bayes Classification Report')

plot_classification_report(y_test, y_pred_gb, 'Gradient Boosting Classification Report')


# ===== Logistic Regression =====

# Preparing valid data and vectorizing
data_valid_fraudulent = data[data['fraudulent'].notnull() & (data['fraudulent'] != 'unknown')].copy()
data_valid_fraudulent['fraudulent_encoded'] = LabelEncoder().fit_transform(data_valid_fraudulent['fraudulent'])
X_valid = TfidfVectorizer().fit_transform(data_valid_fraudulent['Cleaned_company_profile'])
y_valid = data_valid_fraudulent['fraudulent_encoded']


# Splitting and training Logistic Regression
X_train_valid, X_test_valid, y_train_valid, y_test_valid = train_test_split(X_valid, y_valid, test_size=0.2, random_state=42)

log_reg_valid = LogisticRegression(max_iter=1000)
log_reg_valid.fit(X_train_valid, y_train_valid)


# Prediction and evaluation
y_pred_valid = log_reg_valid.predict(X_test_valid)

print("Logistic Regression Confusion Matrix:\n", confusion_matrix(y_test_valid, y_pred_valid))

print("Logistic Regression Classification Report:\n", classification_report(y_test_valid, y_pred_valid))

```